

Hamiltonian Neural Network for Hamiltonian Monte Carlo

February 1, 2025

This paper explains the parts of [Dhulipala et al. \(2023\)](#) that are difficult to understand and presents the replication results obtained using TensorFlow.

0.1 Typos and Explanation

- On page 9, the sentence *"where $\{q^+, p^+\}$ and $\{q^-, p^-\}$ are, respectively, the leftmost and rightmost states of a sub-tree in the binary tree."* What do the leftmost and rightmost states mean in high dimensions? In high dimensions, the right direction corresponds to positive step size dt , while the left direction corresponds to negative step size dt .
- On page 9, what error does the equation (20) measure? The error monitors the integration error. According to the simulation method of u in Algorithm 4, we know that $\log u + H(q(0), p(0)) < 0$. If the integration error is small, it should hold that $\log u + H(q, p) < \Delta_{\max}$. Otherwise, if this inequality is violated, the integration error is very large, and the tree should stop.
- In line 16 of Algorithm 4, there is a typo. It should be ν_j instead of j .
- In Section 3.2, the author does not specify the activation function. In my implementation, I tried "sin", "ReLU", and "tanh". The loss did not decrease at all when using "ReLU," whereas both "sin" and "tanh" produced similar results.
- In Section 7.2, the author does not explicitly mention the target density. Therefore, we need to derive the target density for this elliptic PDE problem ourselves. Given that $u(x, y) = \sin(2x) + \sin(2y)$, the PDE can be written explicitly as:

$$\begin{aligned} & \frac{\partial}{\partial x} \left(k(x, y) \frac{\partial u}{\partial x} \right) + \frac{\partial}{\partial y} \left(k(x, y) \frac{\partial u}{\partial y} \right) \\ &= \frac{\partial k}{\partial x}(x, y) 2 \cos 2x - k(x, y) 4 \sin 2x + \frac{\partial k}{\partial y}(x, y) 2 \cos 2y - k(x, y) 4 \sin 2y. \end{aligned}$$

Let $\{(x_i, y_i), i = 1, \dots, 50\}$ denote the 50 sensor locations, and let $q, h \in \mathbb{R}^{50}$ represent the values of k at these 50 locations (to be inferred) and the observations of f , respectively. The target density can be expressed as:

$$f(q) \propto \exp \left(-\frac{1}{2} \sum_{i=1}^{50} \left(\frac{\hat{\partial} k}{\partial x}(x_i, y_i) 2 \cos 2x_i - q_i 4 \sin 2x_i + \frac{\hat{\partial} k}{\partial y}(x_i, y_i) 2 \cos 2y_i - q_i 4 \sin 2y_i - h_i \right)^2 \right),$$

where $\frac{\hat{\partial} k}{\partial x}$ and $\frac{\hat{\partial} k}{\partial y}$ represent approximations of the partial derivatives. There are various methods to approximate these derivatives; here, we use a simple approach. Assume

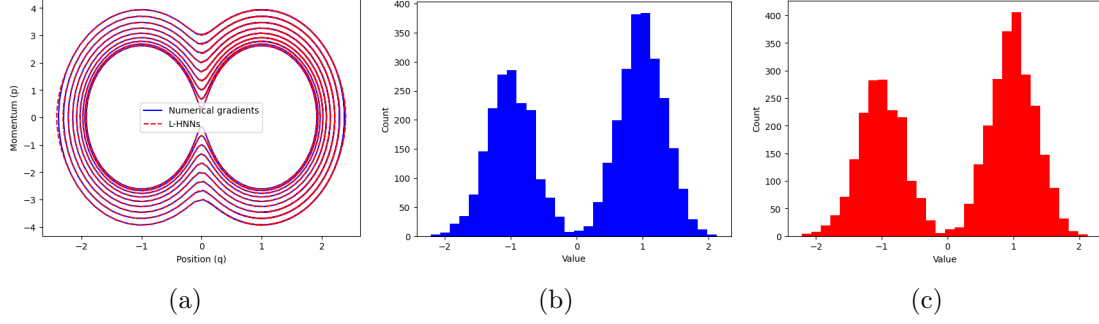


Figure 1: (a) $\{q,p\}$ phase space plot corresponding to different energy levels. (b) and (c) Histogram of samples from a 1-D Gaussian mixture density using traditional HMC and L-HNNs in HMC, respectively.

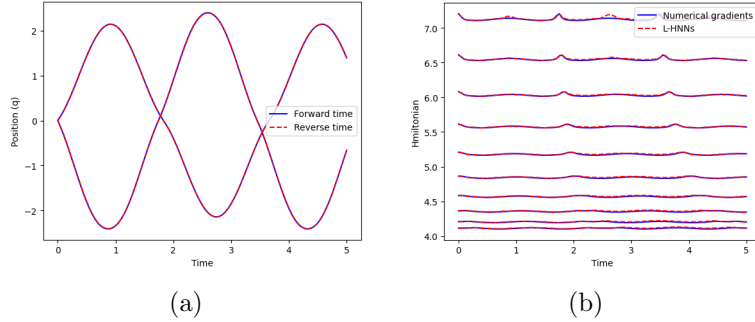


Figure 2: (a) Time reversibility of the L-HNNs in predicting the Hamiltonian dynamics under two different sets of initial conditions, given a 1-D Gaussian mixture density. (b) Conservation of the Hamiltonian for different energy levels given a 1-D Gaussian mixture density.

$k(x, y) = ax + by$. With 50 observations of k , the coefficients (a, b) can be approximated using linear regression:

$$(\hat{a}, \hat{b}) = (X^T X)^{-1} X^T q,$$

where $X \in \mathbb{R}^{50 \times 2}$ is the matrix of stacked sensor values. Substituting these approximations, the target density becomes:

$$f(q) \propto \exp \left(-\frac{1}{2} \sum_{i=1}^{50} \left(\hat{a} 2 \cos 2x_i - q_i 4 \sin 2x_i + \hat{b} 2 \cos 2y_i - q_i 4 \sin 2y_i - h_i \right)^2 \right).$$

0.2 Replication Results

I used TensorFlow to replicate the results presented in [Dhulipala et al. \(2023\)](#). All the figures and tables referenced in this subsection are available in the file "plot_Dhulipala.ipynb".

Sampling under 1D Gaussian mixture I used "sin" as the activation function and tested the L-HNN for HMC on the 1D Gaussian mixture model. The phase space plot and sample histograms are presented in Figure 1, while the time reversibility and Hamiltonian conservation plots are shown in Figure 2. These figures demonstrate that my trained HNN performs comparably to the trained HNN in [Dhulipala et al. \(2023\)](#) on the 1D Gaussian mixture model.

NUTS with L-HNNs Table 1 shows that our trained model performs similarly to [Dhulipala et al. \(2023\)](#). Under both HMC and NUTS, the ESS has significantly improved, and the improvement percentage is comparable to that reported in [Dhulipala et al. \(2023\)](#). From Figure 3, I observed some degeneracy in the path; however, it was not as pronounced as that

Table 1: Comparison of the performance of traditional HMC, traditional NUTS, HNN-HMC, and HNNNUTS, considering a 1-D Gaussian mixture density. ESS per gradient is used as the performance metric.

	ESS per gradient of target density
L-HNNs in HMC	$5.87E - 3$
Traditional HMC	$9.30E - 5$
L-HNNs in NUTS	$4.02E - 3$
Traditional NUTS	$4.03E - 4$

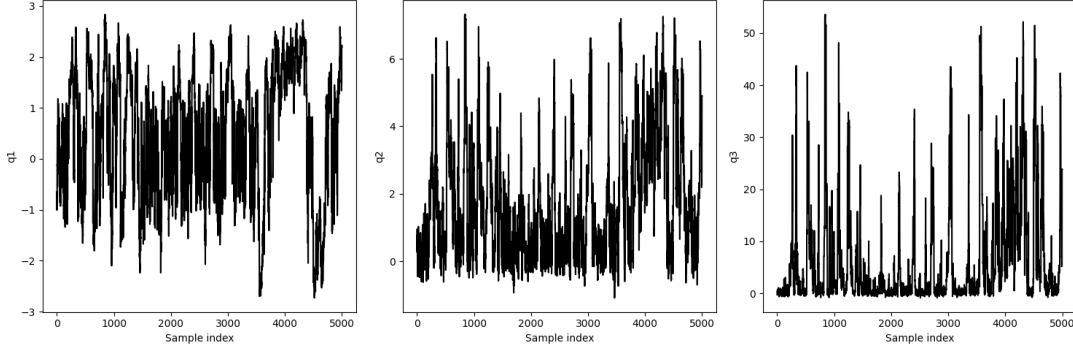


Figure 3: Degeneracy in sampling when using L-HNNs in NUTS for a 3-D Rosenbrock density. These trace plots correspond to the three dimensions q_1 , q_2 , and q_3 .

reported in Dhulipala et al. (2023). This difference could be attributed to randomness, as well as the possibility that they used a less well-trained HNN to intentionally demonstrate stronger degeneracy.

Sampling from a 3-D Rosenbrock density Figure 4 is similar to that in Dhulipala et al. (2023). Figure 5 shows that the integration error of L-HNN is generally smaller than that of HNN; however, the difference is not as significant as reported in Dhulipala et al. (2023). In fact, I believe there is no real difference between L-HNN and HNN. According to the L-HNN formula (Equation (16) in Dhulipala et al. (2023)), given $\mathbf{u}_P$ as the output of a P -layer neural network, and ω_P and $\mathbf{b}_P$ as the weights and biases, the L-HNN is defined as:

$$H_\theta = \mathbf{1}^T \lambda = \mathbf{1}^T (\omega_P \mathbf{u}_P + \mathbf{b}_P) = \mathbf{1}^T \omega_P \mathbf{u}_P + \mathbf{1}^T \mathbf{b}_P,$$

which is mathematically equivalent to an HNN with $\mathbf{1}^T \omega_P$ and $\mathbf{1}^T \mathbf{b}_P$ as the weights and biases of the last layer. Therefore, I believe that Figure 6 in Dhulipala et al. (2023) presents an unfair comparison. Figure 6 and Table 2 are similar to that in Dhulipala et al. (2023).

Analytical case studies. It is worth mentioning that Dhulipala had a typo in the density function of the 2-D Neal’s funnel density. The variance of q_1 should be $3^2 = 9$. Therefore, the correct 2-D Neal’s funnel density should be defined as:

$$f(\mathbf{q}) \propto \begin{cases} q_1 = \mathcal{N}(0, 9) \\ q_2 = \mathcal{N}(0, \exp^{q_1}) \end{cases}.$$

In the implementation, the correct density is used. L-HNNs are trained on a sampling of the 2-D Neal’s funnel density, the 5-D ill-conditioned Gaussian density, and the 10-D degenerate Rosenbrock density. Table 3 and Figures 7 and 8 show results similar to those reported in Dhulipala et al. (2023).

Computational case studies Table 4 and Figure 9 demonstrate that our results are consistent with those reported in Dhulipala et al. (2023).

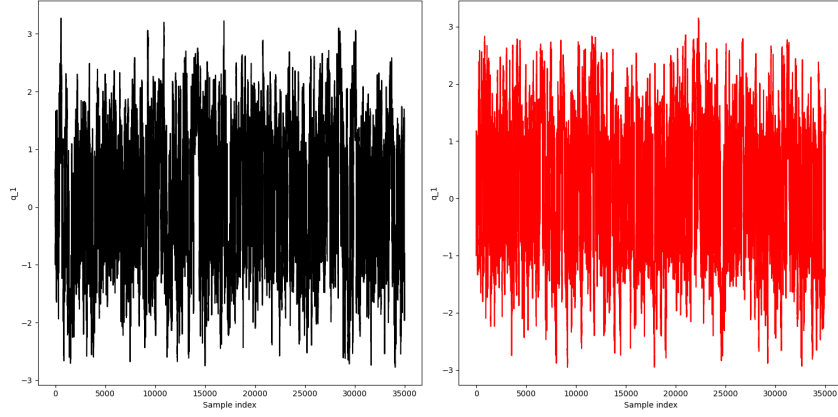


Figure 4: Trace plots of dimension q_1 for a 3-D Rosenbrock density simulated via efficient NUTS with online error monitoring using (left) HNNs and (right) L-HNNs.

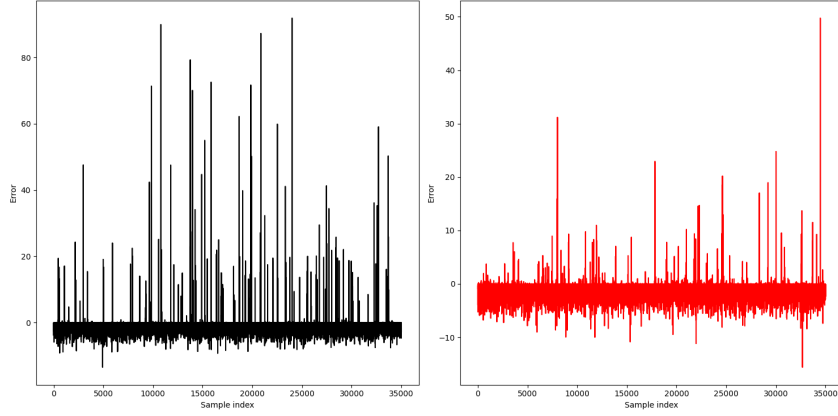


Figure 5: L-HNNs error metric, as defined by the quantity $\varepsilon \equiv H(q', p') + \log u$ in Algorithm 5, while generating samples from a 3-D Rosenbrock density using (left) HNNs and (right) L-HNNs in efficient NUTS with online error monitoring.

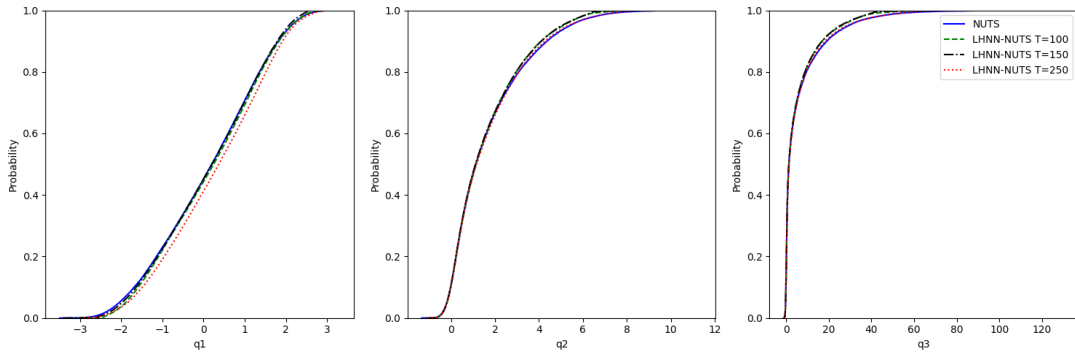


Figure 6: Empirical CDF plots of samples from a 3-D Rosenbrock density simulated via NUTS and L-HNNs in NUTS with online error monitoring, using different training datasets. eCDF plots are presented for the dimensions q_1 , q_2 , and q_3 .

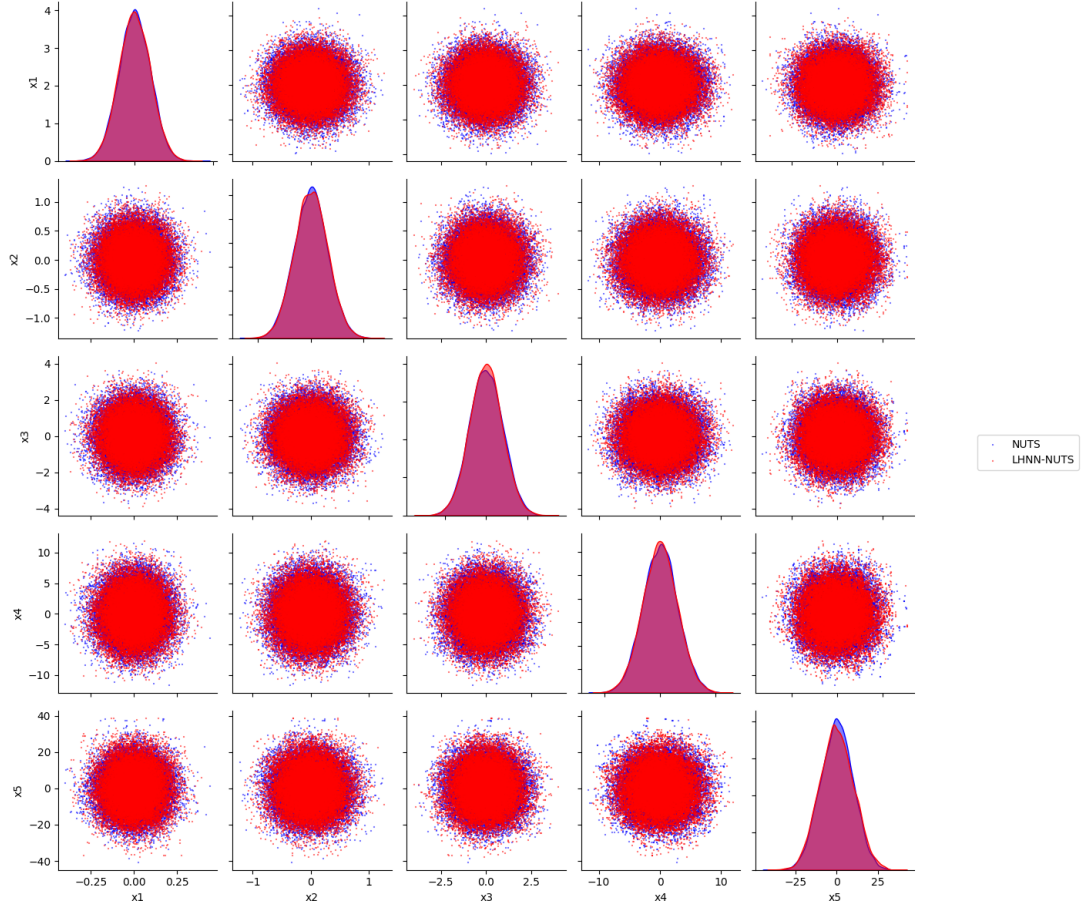


Figure 7: Scatter plot comparison between NUTS and LHNN-NUTS with online error monitoring for a 5-D ill-conditioned Gaussian distribution. (Blue dots: traditional NUTS; Red dots: LHNN-NUTS with online error monitoring)

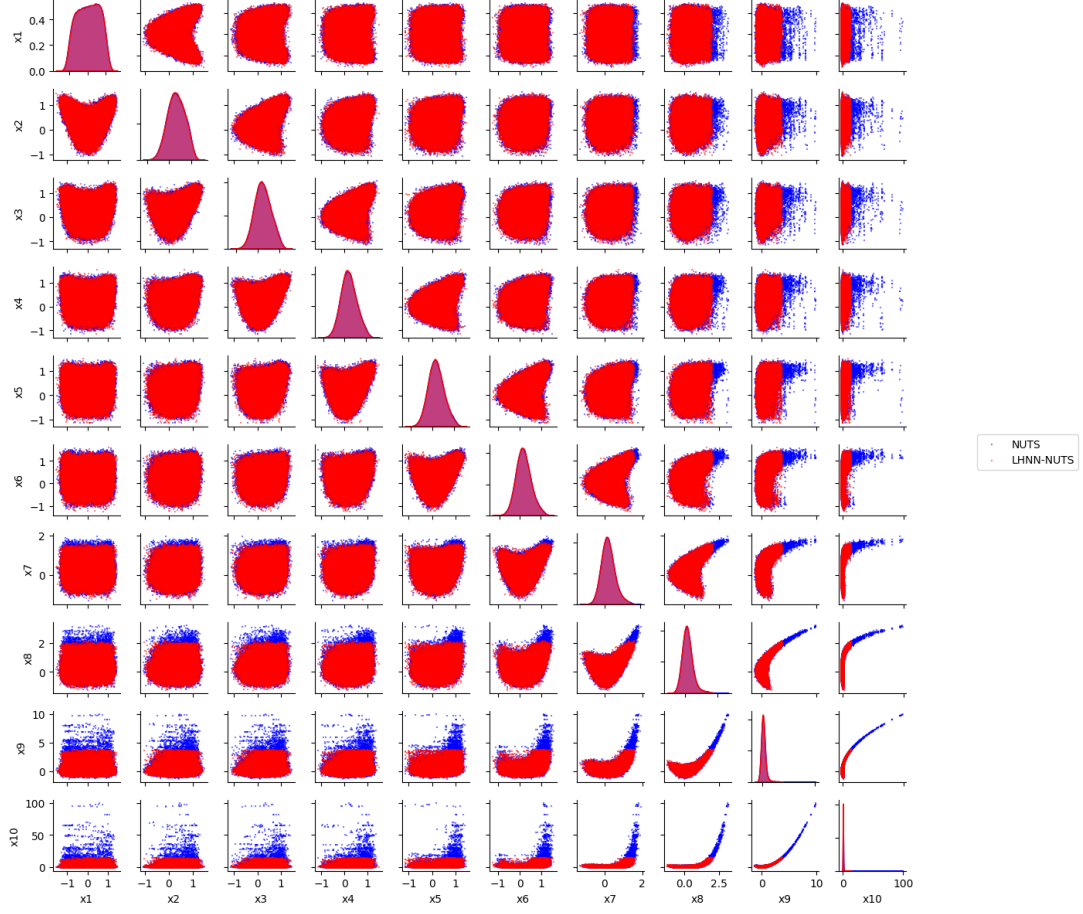


Figure 8: Scatter plot comparison between NUTS and LHNN-NUTS with online error monitoring, considering a 10-D degenerate Rosenbrock distribution. (Blue dots: traditional NUTS; Red dots: LHNN-NUTS with online error monitoring)

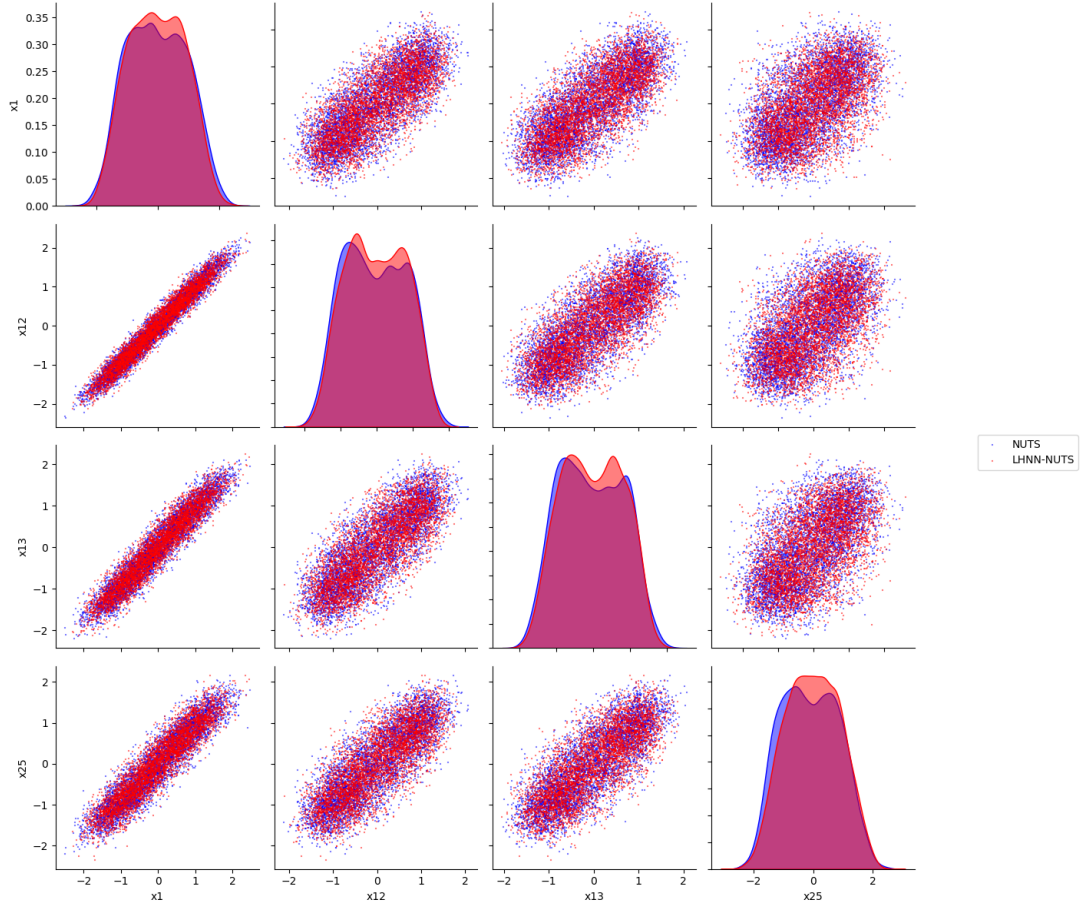


Figure 9: Sampling results for four of the 25 dimensions, namely, 1, 12, 13, and 25, of the Allen-Cahn equation. It is observed that the LHNN-NUTS and traditional NUTS results compare well with each other.

Table 2: Performance comparison between traditional NUTS and HNN-NUTS with different training data characteristics, considering a 3-D degenerate Rosenbrock density.

	ESS	# gradients of target density	Avg. ESS per gradient of target density
NUTS	(1306.86, 1968.87, 2074.03)	13,004,142	0.000137
$M_t = 40, T = 100$ LHNN-NUTS 1	(1509.69, 2355.79, 1962.16)	1,105,447 Training: 160,000 Evaluation: 945,447	0.001757
$M_t = 40, T = 150$ LHNN-NUTS 2	(1378.65, 2084.02, 1990.07)	837,691 Training: 240,000 Evaluation: 597,691	0.002170
$M_t = 40, T = 250$ LHNN-NUTS 3	(1547.13, 2156.33, 2327.47)	413,840 Training: 400,000 Evaluation: 13,840	0.004858

Table 3: Summary of the performance of LHNN-NUTS with online error monitoring across the different analytical case studies.

	ESS	# gradients of target density	Avg. ESS per gradient of target density
2-D Neal's funnel			
LHNN-NUTS	()	Training: 400,000 Evaluation:	
NUTS	(697.86, 503.90)	13,625,586	0.000044
5-D ill-conditioned Gaussian			
LHNN-NUTS	(20000.02, 18144.69, 14207.53, 8023.24, 2472.59)	425,000 Training: 400,000 Evaluation: 25,000	0.0295
NUTS	(20000, 17903.15, 14658.81, 8185.42, 2756.69)	9,133,676	0.00139
10-D degenerate Rosenbrock			
LHNN-NUTS	(15447.20, 29445.31, 27893.89, 25361.00, 21654.22, 13608.50, 8473.87, 5864.44, 3544.28, 2039.00)	400,000 Training: 400,000 Evaluation: 0	0.0383
NUTS	(15951.39, 27579.14, 23974.31, 19612.90, 13089.86, 8263.75, 5217.63, 3198.49, 1654.23, 1227.98)	6,761,660	0.00177

References

Dhulipala, S. L., Y. Che, and M. D. Shields (2023). Efficient bayesian inference with latent hamiltonian neural networks in no-u-turn sampling. *Journal of Computational Physics* 492, 112425.

Table 4: Summary of the performance of LHNN-NUTS with online error monitoring across the different computational case studies.

	ESS	# gradients of target density	Avg. ESS per gradient of target density
Allen-Cahn stochastic PDE (25 inference parameters)			
LHNN-NUTS	289	7,272 Training: 10,000 Evaluation: 17,272	0.016736
NUTS	232.42	581,752	0.00040
Elliptic PDE (50 inference parameters)			
LHNN-NUTS		Training: 64,000 Evaluation:	
NUTS			